



INFORME DE LA GUÍA PRÁCTICA

I. PORTADA

Tema:	Gestión de colección de datos utilizando vectores
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Camacho Monta Josue Jampier Chalco Tasna Kenneth Mateo Cunalata Mendoza Damian Alexander Silva Camuendo Luis Alexander Tacuri Santillan Monica Sara Tisalema Guashco Darwin Joel
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Determinar el cálculo de la media de valores estructurados utilizando programación orientada a objetos y vectores.

Específicos:

- Diseñar las clases y atributos definiendo la estructura orientada a objetos para los datos cuantitativos del TDA.
- Implementar el vector estático para encapsular y gestionar el almacenamiento de los elementos dentro del TDA.
- Calcular la media de los valores recorriendo el vector mediante la extracción de los datos cuantitativos de los objetos.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

Implemente una aplicación para la gestión de los datos de los estudiantes matriculados en un curso con cupo para hasta 20 estudiantes. Para ello, deberá:

1. Definir la clase Estudiante, con atributos cédula de identidad, nombres, apellidos, fecha de nacimiento y un vector con las notas (máximo de 7 notas, asumiendo que es la cantidad máxima de actividades en que se evaluará a un estudiante).
2. Implementar adecuadamente el constructor de la clase Estudiante y un método buscar (), en la clase que considere oportuno, para que busque por cédula a un estudiante dado.
3. Implemente el siguiente menú de opciones en la aplicación Java por consola:
=== GESTOR DE PERSONAS ===

- 1.- Estudiantes.
- 2.- Registro de calificaciones.
- 3.- Determinar el promedio de notas de un estudiante.
- 4.- Determinar el promedio de notas del curso. Teclee su opción (1-4)

El comportamiento del programa cuando el usuario interactúe con el menú de opciones deberá ser como se describe a continuación. En la opción 1, al usuario se le mostrará un listado con un autonumérico, para identificar cada registro de cada estudiante, y los datos de cada estudiante previamente registrado. A partir de ello, el usuario podrá acceder a un submenú que le permita ingresar, modificar y eliminar los datos de un estudiante. Cada



vez que un usuario inserte/modifique/elimine un registro, si es que tiene sentido, se le preguntará si desea realizar la misma acción una vez más. Si la cantidad máxima de estudiantes ya fue registrada, no se permitirá insertar datos de un nuevo estudiante. Si no hay estudiantes registrados, no se le permitirá intentar eliminar el registro de un estudiante. En todo caso, para las opciones de modificar y eliminar, el usuario comenzará indicando el valor de autonumérico que identifique al registro de estudiante que se quiere modificar/eliminar.

En la opción 2, el usuario podrá ingresar los datos de calificaciones de un estudiante. Comenzará introduciendo el número de cédula del estudiante en cuestión. Si el número de cédula pertenece a uno de los estudiantes registrados, se mostrarán sus datos (nombres y apellidos y edad), y se permitirá ingresar tantos valores de calificaciones como el usuario considere. Se le mostrará un listado de calificaciones previamente registradas. Se permitirá modificar o eliminar cada una de estas, o insertar nuevas calificaciones. Si se llega a ingresar la cantidad máxima de calificaciones posible, el sistema notificará que se han ingresado todas las calificaciones posibles y se da por terminado el proceso de entrada de calificaciones. Si el número de cédula ingresado inicialmente no fuera el de alguno de los estudiantes registrados, se le notificará oportunamente al usuario y se le dará la opción de ingresar otro número de cédula o regresar al menú principal.

En la opción 3, el usuario comenzará introduciendo el número de cédula del estudiante del que se quiere calcular el promedio de calificaciones. Si el número de cédula pertenece a uno de los estudiantes ya registrados, se mostrarán sus datos (nombres y apellidos, edad y promedio de calificaciones). Caso contrario, se emitirá un mensaje de error indicando que no se encontró un estudiante con el número de cédula indicado.

En la opción 4, se mostrará un mensaje indicando el valor del promedio general de calificaciones. Si ningún estudiante tiene calificaciones registradas, de igual manera, se indicará que No se han registrado calificaciones de estudiantes.

Nota: el conjunto de estudiantes, y el conjunto de notas por estudiante, se almacenará en vectores.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial
- TAC
- PC de escritorio o portátil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

Implementación de aplicación para la gestión de los datos de los estudiantes matriculados en un curso con cupo para hasta 20 estudiantes.

2.7 Resultados obtenidos

Resultados obtenidos del TDA vehículos.

1. Ejercicio

Desarrollar en Java y C++ un programa que registre información de vehículos usando:



- Un arreglo estático o de tamaño fijo;
- Al menos dos tipos de datos primitivos;
- Una clase;
- Instanciación de objetos;
- Herencia;
- Polimorfismo;
- Comentarios que identifiquen cada concepto.

2. Solución Propuesta

Se crea un TDA GestorVehiculos / RegistroVehiculos con capacidad para diez elementos. El arreglo almacena referencias/punteros de tipo Vehiculo, pero los objetos reales pueden ser automóviles o motocicletas.

El programa contiene las siguientes clases y módulos organizados por responsabilidades:

Clase / Módulo	Responsabilidad
Vehiculo	Clase abstracta que contiene los atributos comunes.
Automovil	Clase hija con número de puertas y estado eléctrico.
Motocicleta	Clase hija con cilindrada y maletero.
IVehiculoGestor	Interfaz / Clase base abstracta pura que define el contrato de operaciones.
GestorVehiculos	TDA que administra el arreglo estático de tamaño fijo (capacidad 10).
MainVehiculos / Main	Presenta el menú interactivo, valida entradas, recibe datos e instancia objetos.

3. ¿Qué es un TDA?

Un Tipo de Dato Abstracto define:

- Los datos que puede almacenar;
- Las operaciones que pueden realizarse;
- Las reglas que siempre deben cumplirse.

En este ejercicio, el TDA está representado por GestorVehiculos:

En Java:

```
public class GestorVehiculos implements IVehiculoGestor {  
    private static final int CAPACIDAD = 10;  
    private final Vehiculo[] vehiculos = new Vehiculo[CAPACIDAD];  
    private int cantidad;  
}
```

En C++:

```
class GestorVehiculos : public IVehiculoGestor {
```



```
private:
    static const int CAPACIDAD = 10;
    std::unique_ptr<Vehiculo> vehiculos[CAPACIDAD];
    int cantidad;
};
```

Los datos están encapsulados porque vehiculos y cantidad son privados. Para trabajar con ellos se utilizan métodos públicos:

```
gestor.registrar(nuevoVehiculo);
gestor.mostrarTodos();
gestor.estaLleno();
```

4. Relación entre Requisitos y Código

Requisito	Aplicación en Java	Aplicación en C++
Arreglo estático	new Vehiculo[10] crea un arreglo de tamaño fijo.	std::unique_ptr<Vehiculo> vehiculos[10] estático en capacidad.
Datos primitivos	int, double y boolean.	int, double y bool.
Clase	Vehiculo, Automovil, Motocicleta, GestorVehiculos.	Vehiculo, Automovil, Motocicleta, GestorVehiculos.
Instanciación	new Automovil(...), new Motocicleta(...).	std::make_unique<Automovil>(...).
Herencia	Automovil extends Vehiculo.	class Automovil : public Vehiculo.
Polimorfismo	Método abstracto sobrescrito con @Override.	Método virtual puro sobrescrito con override.
Encapsulamiento	Atributos private y protected.	Atributos private y protected.
Comentarios	Explicaciones detalladas en cada método.	Explicaciones explícitas de gestión de memoria y herencia.

5. Paso 1: Importar / Incluir bibliotecas de lectura y entrada/salida

En Java:

```
import java.util.Scanner;
```

Scanner permite leer los datos ingresados desde el teclado.

En C++:

```
#include <iostream>
#include <string>
#include <limits>
```



#include <memory>

std::cin y std::cout permiten manejar la entrada y salida estándar, mientras que std::unique_ptr facilita el manejo automático de memoria.

6. Paso 2: Crear la clase abstracta Vehiculo

En Java:

```
public abstract class Vehiculo {
    protected String placa;
    protected String marca;
    protected String modelo;
    protected int anio;
    protected double precio;
    protected boolean disponible;

    public abstract void mostrarInformacion();
}
```

En C++:

```
class Vehiculo {
protected:
    std::string placa;
    std::string marca;
    std::string modelo;
    int anio;
    double precio;
    bool disponible;

public:
    virtual ~Vehiculo() = default;
    virtual void mostrarInformacion() const = 0;
};
```

- **abstract / virtual ... = 0** impide instanciar vehículos genéricos.
- **String / std::string** almacena texto (tipo de referencia / objeto).
- **int, double y boolean / bool** son tipos de datos primitivos.
- **protected** permite que las clases hijas utilicen directamente los atributos de la clase padre.
- **mostrarInformacion()** no tiene implementación en la clase base porque cada subclase la define según su propia estructura.

7. Paso 3: Definir las clases derivadas Automovil y Motocicleta (Herencia)

En Java:

```
public class Automovil extends Vehiculo {
    private int numeroPuertas;
    private boolean electrico;

    public Automovil(String placa, String marca, String modelo, int anio, double precio,
boolean disponible, int numeroPuertas, boolean electrico) {
        super(placa, marca, modelo, anio, precio, disponible);
        this.numeroPuertas = numeroPuertas;
        this.electrico = electrico;
    }
}
```



```
}  
  
public class Motocicleta extends Vehiculo {  
    private int cilindrada;  
    private boolean tieneMaletero;  
  
    public Motocicleta(String placa, String marca, String modelo, int anio, double precio,  
        boolean disponible, int cilindrada, boolean tieneMaletero) {  
        super(placa, marca, modelo, anio, precio, disponible);  
        this.cilindrada = cilindrada;  
        this.tieneMaletero = tieneMaletero;  
    }  
}
```

En C++:

```
class Automovil : public Vehiculo {  
private:  
    int numeroPuertas;  
    bool electrico;  
  
public:  
    Automovil(const std::string& placa, const std::string& marca, const std::string& modelo,  
        int anio, double precio, bool disponible, int numeroPuertas, bool electrico)  
        : Vehiculo(placa, marca, modelo, anio, precio, disponible),  
        numeroPuertas(numeroPuertas), electrico(electrico) {}  
};  
  
class Motocicleta : public Vehiculo {  
private:  
    int cilindrada;  
    bool tieneMaletero;  
  
public:  
    Motocicleta(const std::string& placa, const std::string& marca, const std::string& modelo,  
        int anio, double precio, bool disponible, int cilindrada, bool tieneMaletero)  
        : Vehiculo(placa, marca, modelo, anio, precio, disponible),  
        cilindrada(cilindrada), tieneMaletero(tieneMaletero) {}  
};
```

La palabra reservada `extends` en Java o la sintaxis `: public Vehiculo` en C++ establece la herencia. Ambas clases reutilizan los atributos y comportamientos comunes de la clase padre.

8. Paso 4: Aplicar polimorfismo

Cada clase hija sobrescribe el método abstracto de la clase base:

En Java:

```
@Override  
public void mostrarInformacion() {  
    System.out.println("-----");  
    System.out.println("Tipo: Automóvil");  
    mostrarDatosComunes();  
    System.out.println("Número de Puertas: " + numeroPuertas);  
    System.out.println("Eléctrico: " + (electrico ? "Sí" : "No"));  
}
```



```
System.out.println("-----");  
}
```

En C++:

```
void Automovil::mostrarInformacion() const override {  
    std::cout << "-----" << std::endl;  
    std::cout << "Tipo: Automovil" << std::endl;  
    mostrarDatosComunes();  
    std::cout << "Numero de Puertas: " << numeroPuertas << std::endl;  
    std::cout << "Electrico: " << (electrico ? "Si" : "No") << std::endl;  
    std::cout << "-----" << std::endl;  
}
```

Al iterar en el TDA:

```
vehiculos[i].mostrarInformacion(); // Java  
vehiculos[i]->mostrarInformacion(); // C++
```

La referencia/puntero pertenece a Vehiculo, pero el entorno de ejecución identifica el objeto real y selecciona automáticamente la implementación correspondiente de Automovil o Motocicleta. Esto constituye el polimorfismo en tiempo de ejecución.

9. Paso 5: Crear el arreglo estático

Código en Java:

```
private static final int CAPACIDAD = 10;  
private final Vehiculo[] vehiculos = new Vehiculo[CAPACIDAD];
```

Código en C++:

```
static const int CAPACIDAD = 10;  
std::unique_ptr<Vehiculo> vehiculos[CAPACIDAD];
```

En este contexto, "estático" significa que el arreglo posee una capacidad fija predefinida. Después de instanciarlo, su longitud no puede alterarse.

10. Paso 6: Registrar vehículos en el TDA

Java:

```
@Override  
public boolean registrar(Vehiculo nuevoVehiculo) {  
    if (nuevoVehiculo == null || estaLleno()) {  
        return false;  
    }  
    if (buscarPorPlaca(nuevoVehiculo.getPlaca()) != -1) {  
        return false;  
    }  
    vehiculos[cantidad] = nuevoVehiculo;  
    cantidad++;  
    return true;  
}
```

C++:

```
bool GestorVehiculos::registrar(std::unique_ptr<Vehiculo> nuevoVehiculo) {
```



```
if (!nuevoVehiculo || estaLleno()) {  
    return false;  
}  
if (buscarPorPlaca(nuevoVehiculo->getPlaca()) != -1) {  
    return false;  
}  
vehiculos[cantidad] = std::move(nuevoVehiculo);  
cantidad++;  
return true;  
}
```

El vehículo se guarda en la posición indicada por el contador cantidad, asegurando que la memoria se mantenga contigua y sin huecos intermediarios.

11. Paso 7: Instanciar objetos de clases derivadas

Java:

```
Vehiculo nuevoVehiculo = new Automovil(placa, marca, modelo, anio, precio, disponible,  
puertas, electrico);  
gestor.registrar(nuevoVehiculo);
```

C++:

```
auto auto_ = std::make_unique<Automovil>(placa, marca, modelo, anio, precio, disponible,  
puertas, electrico);  
gestor.registrar(std::move(auto_));
```

La operación ejecuta tres acciones fundamentales:

- Creación e intermediación del objeto en memoria dinámica.
- Ejecución del constructor para inicializar atributos heredados y propios.
- Asignación de la referencia/puntero base para tratamiento polimórfico.

12. Tipos de datos utilizados

Variables	Tipo	Categoría	Descripción / Uso
anio, cantidad, numeroPuertas, cilindrada	int	Primitivo	Valores numéricos enteros sin parte decimal.
precio	double	Primitivo	Valores numéricos con coma flotante de doble precisión.
disponible, electrico, tieneMaletero	boolean / bool	Primitivo	Valores lógicos (verdadero o falso).
placa, marca, modelo	String / std::string	Referencia / No primitivo	Cadenas de caracteres para texto.
vehiculos[]	Vehiculo[] / unique_ptr[]	Referencia / No primitivo	Arreglo de referencias o punteros a objetos Vehiculo.



13. Diagramas de clase

classDiagram

```
class Vehiculo {
    #String placa
    #String marca
    #String modelo
    #int anio
    #double precio
    #boolean disponible
    +mostrarInformacion()*
}
class Automovil {
    -int numeroPuertas
    -boolean electrico
    +mostrarInformacion()
}
class Motocicleta {
    -int cilindrada
    -boolean tieneMaletero
    +mostrarInformacion()
}
class RegistroVehiculos {
    -Vehiculo[] vehiculos
    -int cantidad
    +registrar(Vehiculo v) boolean
    +mostrarTodos()
}
```

Vehiculo <|-- Automovil

Vehiculo <|-- Motocicleta

RegistroVehiculos o—Vehiculo

14. Especificación del TDA

Estado:

- **vehiculos:** Arreglo fijo de diez referencias o punteros a objetos de tipo Vehiculo.
- **cantidad:** Número de posiciones actualmente ocupadas (entre 0 y 10).
- **CAPACIDAD:** Constante entera con valor fijado en 10.

Operaciones:

Método	Resultado / Descripción
registrar()	Agrega un vehículo si existe espacio disponible y la placa no está duplicada.
mostrarTodos()	Presenta la información de todos los objetos registrados mediante polimorfismo.
buscarPorPlaca()	Comprueba si una placa ya existe y devuelve su índice numérico o -1.
modificar()	Actualiza el precio y la disponibilidad del vehículo en una posición específica.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Método	Resultado / Descripción
eliminar()	Remueve un vehículo del arreglo y realiza la compactación contigua del vector.
estaLleno()	Indica si las diez posiciones del arreglo se encuentran totalmente ocupadas.
getCantidad()	Devuelve el número total de registros activos.

Invariantes del TDA:

1. $0 \leq \text{cantidad} \leq \text{CAPACIDAD}$.
2. Las posiciones desde 0 hasta cantidad - 1 contienen objetos válidos.
3. Las posiciones desde cantidad hasta CAPACIDAD - 1 están nulas/vacías (null o reset()).
4. No existen placas repetidas dentro del sistema.
5. El año de fabricación es mayor a 0 (entre 1886 y 2100).
6. El precio de venta no es negativo (precio ≥ 0.0).
7. Un automóvil posee un número positivo de puertas.
8. Una motocicleta posee una cilindrada mayor a 0 cc.

15. Diagrama de flujo del programa principal

INICIO

instanciar RegistroVehiculos

REPETIR

mostrar menú

leer opción

SEGÚN opción HACER

- 1:
leer datos del automóvil
instanciar Automovil[cite: 1]
registrar objeto[cite: 1]
- 2:
leer datos de la motocicleta
instanciar Motocicleta[cite: 1]
registrar objeto[cite: 1]
- 3:
recorrer arreglo[cite: 1]
mostrar cada objeto mediante polimorfismo[cite: 1]
- 4:
finalizar

FIN SEGÚN



16. Prueba de escritorio

Paso	Operación realizada	Atributo cantidad	Resultado observado / Estado del TDA
1	Crear el registro de vehículos	0	Las 10 posiciones contienen nulos / punteros limpios.
2	Registrar Automóvil Toyota (Placa "PBA-1234")	1	Objeto Automovil guardado exitosamente en vehiculos[0].
3	Registrar Motocicleta Yamaha (Placa "HA-567B")	2	Objeto Motocicleta guardado exitosamente en vehiculos[1].
4	Ejecutar mostrarTodos()	2	Se ejecutan dinámicamente dos métodos sobrescritos polimórficos distintos.
5	Intentar registrar vehículo con placa repetida "PBA-1234"	2	El registro es rechazado por la validación de unicidad de placa.
6	Modificar precio de "HA-567B" a \$2,500.00	2	Búsqueda por placa retorna índice 1, actualizando el estado de la moto.
7	Eliminar vehículo "PBA-1234"	1	Se elimina vehiculos[0] y la moto en vehiculos[1] se desplaza a vehiculos[0].

Resultados obtenidos del TDA estudiante.

17. Ejercicio

Implementar una aplicación para la gestión de los datos de los estudiantes matriculados en un curso con cupo para hasta 20 estudiantes. Para ello, deberá:

- Definir la clase Estudiante, con atributos cédula de identidad, nombres, apellidos, fecha de nacimiento y un vector con las notas (máximo de 7 notas, asumiendo que es la cantidad máxima de actividades en que se evaluará a un estudiante).
- Implementar adecuadamente el constructor de la clase Estudiante y un método buscar(), en la clase que considere oportuno, para que busque por cédula a un estudiante dado.
- Implementar el siguiente menú de opciones en consola:

Gestor persona:

- Estudiantes.
 - Registro de calificaciones.
 - Determinar el promedio de notas de un estudiante.
 - Determinar el promedio de notas del curso.
- Garantizar el comportamiento del submenú para ingresar, modificar y eliminar estudiantes respetando el uso de autonuméricos, límites de cupos (20 estudiantes), validaciones de estado, ingreso repetitivo interactivo, gestión dinámica de calificaciones (máximo 7 notas) y cálculo general de promedios.



18. Solución Propuesta

Se diseña el TDA RegistroEstudiantes (o GestorEstudiantes) con una capacidad fija de 10 elementos. El arreglo almacena referencias/punteros a la clase base abstracta Estudiante, cuyos objetos reales corresponden a las subclases derivadas EstudiantePregrado y EstudiantePosgrado.

Clase / Módulo	Responsabilidad
Estudiante	Clase abstracta base que encapsula los datos y comportamientos comunes.
EstudiantePregrado	Clase derivada con créditos aprobados y nivel/semestre.
EstudiantePosgrado	Clase derivada con título de grado previo y horas de investigación.
RegistroEstudiantes	TDA que administra el arreglo estático, búsquedas, eliminaciones por desplazamiento y capacidad.
Main	Punto de entrada con menú interactivo por consola para Java y C++.

19. ¿Qué es un TDA?

Un Tipo de Dato Abstracto (TDA) define:

1. Los datos almacenados y su encapsulamiento;
2. Las operaciones públicas disponibles sobre ellos;
3. Las reglas e invariantes del sistema.

En Java:

```
public class RegistroEstudiantes {  
    private static final int CAPACIDAD = 10;  
    private final Estudiante[] estudiantes = new Estudiante[CAPACIDAD];  
    private int cantidad;  
}
```

En C++:

```
#include <memory>
```

```
class RegistroEstudiantes {  
private:  
    static const int CAPACIDAD = 10;  
    std::unique_ptr<Estudiante> estudiantes[CAPACIDAD];  
    int cantidad;  
};
```



20. Relación entre Requisitos y Código

Requisito	Aplicación en Java	Aplicación en C++
Arreglo estático	<code>new Estudiante[10]</code>	<code>std::unique_ptr<Estudiante> estudiantes[10]</code>
Datos primitivos	<code>int, double, boolean</code>	<code>int, double, bool</code>
Clase Abstracta	<code>abstract class Estudiante</code>	<code>class Estudiante con virtual ... = 0</code>
Instanciación	<code>new EstudiantePregrado (...)</code>	<code>std::make_unique<EstudiantePregrado> (...)</code>
Herencia	<code>class EstudiantePregrado extends Estudiante</code>	<code>class EstudiantePregrado : public Estudiante</code>
Polimorfismo	<code>@Override public void mostrarInformacion()</code>	<code>void mostrarInformacion() const override</code>

21. Paso 1: Bibliotecas e Input/Output

Para el manejo de datos por consola, en Java utilizamos la clase `Scanner` proveniente del paquete `java.util`, la cual abstrae la lectura desde el flujo estándar `System.in`:

En Java:

```
import java.util.Scanner;
```

```
// Scanner permite capturar entradas del usuario desde la consola estándar  
Scanner scanner = new Scanner(System.in);
```

En C++:

```
#include <iostream>  
#include <string>  
#include <limits>  
#include <memory>
```

```
// std::cin y std::cout gestionan la entrada/salida estándar.  
// std::unique_ptr administra la memoria dinámica automáticamente sin fugas (memory leaks).
```

En C++, se incluyen las librerías estándar `<iostream>` para flujos de entrada/salida (`std::cin` y `std::cout`), `<string>` para la manipulación de cadenas de texto y `<memory>` para la gestión de memoria mediante punteros inteligentes (`std::unique_ptr`).

22. Paso 2: Crear la Clase Abstracta Estudiante



En Java:

// CONCEPTO: CLASE ABSTRACTA Y ENCAPSULAMIENTO

```
public abstract class Estudiante {
    protected String cedula;
    protected String nombre;
    protected String carrera;
    protected int edad;      // Tipo primitivo int
    protected double promedio; // Tipo primitivo double
    protected boolean activo; // Tipo primitivo boolean

    public Estudiante(String cedula, String nombre, String carrera, int edad, double promedio,
boolean activo) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.carrera = carrera;
        this.edad = edad;
        this.promedio = promedio;
        this.activo = activo;
    }

    public String getCedula() { return cedula; }

    // CONCEPTO: MÉTODOS ABSTRACTOS PARA POLIMORFISMO
    public abstract void mostrarInformacion();
    public abstract String getTipoEstudiante();
}
```

Una clase abstracta define la interfaz y el comportamiento común de una jerarquía de clases. No puede instanciarse directamente y sirve como contrato para sus clases derivadas.

En C++:

// CONCEPTO: CLASE ABSTRACTA (con destructor virtual y métodos virtuales puros)

```
class Estudiante {
protected:
    std::string cedula;
    std::string nombre;
    std::string carrera;
    int edad;      // Tipo primitivo int
    double promedio; // Tipo primitivo double
    bool activo;   // Tipo primitivo bool

public:
    Estudiante(const std::string& c, const std::string& n, const std::string& car, int e, double p,
bool a)
        : cedula(c), nombre(n), carrera(car), edad(e), promedio(p), activo(a) {}

    virtual ~Estudiante() = default; // Destructor virtual imprescindible

    std::string getCedula() const { return cedula; }

    // METODOS VIRTUALES PUROS
    virtual void mostrarInformacion() const = 0;
    virtual std::string getTipoEstudiante() const = 0;
}
```



};

En C++, una clase es abstracta si contiene al menos un método virtual puro (= 0). Además, es fundamental definir un destructor virtual para garantizar la correcta liberación de memoria en las clases hijas.

23. Paso 3: Aplicar Herencia

La herencia permite que clases especializadas (EstudiantePregrado y EstudiantePosgrado) extiendan los atributos y métodos de la clase base Estudiante.

En Java:

```
// CONCEPTO: HERENCIA EN JAVA (extends)
public class EstudiantePregrado extends Estudiante {
    private int creditosAprobados;
    private int semestre;

    public EstudiantePregrado(String cedula, String nombre, String carrera, int edad, double
promedio, boolean activo, int creditosAprobados, int semestre) {
        super(cedula, nombre, carrera, edad, promedio, activo);
        this.creditosAprobados = creditosAprobados;
        this.semestre = semestre;
    }
}
```

En C++:

```
// CONCEPTO: HERENCIA EN C++ (: public Base)
class EstudiantePregrado : public Estudiante {
private:
    int creditosAprobados;
    int semestre;

public:
    EstudiantePregrado(const std::string& c, const std::string& n, const std::string& car, int e,
double p, bool a, int cred, int sem)
        : Estudiante(c, n, car, e, p, a), creditosAprobados(cred), semestre(sem) {}
};
```

24. Paso 4: Aplicar polimorfismo

Cada clase hija sobrescribe el método abstracto de la clase base:

En Java:

```
@Override
public void mostrarInformacion() {
    System.out.println("Pregrado | Cédula: " + cedula + " | Nombre: " + nombre + " | Promedio:
" + promedio + " | Créditos: " + creditosAprobados);
}
```

En C++:

```
void mostrarInformacion() const override {
    std::cout << "Pregrado | Cédula: " << cedula << " | Nombre: " << nombre << " | Promedio:
" << promedio << " | Créditos: " << creditosAprobados << std::endl;
}
```



Al iterar en el TDA:

```
estudiantes[i].mostrarInformacion(); // Java  
estudiantes[i]->mostrarInformacion(); // C++
```

La referencia/puntero pertenece a Estudiante, pero el entorno de ejecución identifica el objeto real y selecciona automáticamente la implementación correspondiente de EstudiantePregrado o EstudiantePosgrado. Esto constituye el polimorfismo en tiempo de ejecución.

25. Paso 5: Arreglo Estático de la Estructura TDA

Código en Java:

```
private static final int CAPACIDAD = 10;  
private final Estudiante[] estudiantes = new Estudiante[CAPACIDAD];
```

Código en C++:

```
static const int CAPACIDAD = 10;  
std::unique_ptr<Estudiante> estudiantes[CAPACIDAD];
```

En este contexto, "estático" significa que el arreglo posee una capacidad fija predefinida. Después de instanciarlo, su longitud no puede alterarse.

26. Paso 6: Registrar y Desplazamiento (Shift) en el TDA

Java:

```
public boolean registrar(Estudiante nuevoEstudiante) {  
    if (estaLleno() || existeCedula(nuevoEstudiante.getCedula())) return false;  
    estudiantes[cantidad++] = nuevoEstudiante;  
    return true;  
}
```

```
public boolean eliminar(String cedula) {  
    int idx = buscar(cedula);  
    if (idx == -1) return false;  
    for (int i = idx; i < cantidad - 1; i++) {  
        estudiantes[i] = estudiantes[i + 1];  
    }  
    estudiantes[--cantidad] = null;  
    return true;  
}
```

C++:

```
bool registrar(std::unique_ptr<Estudiante> nuevoEstudiante) {  
    if (estaLleno() || existeCedula(nuevoEstudiante->getCedula())) return false;  
    estudiantes[cantidad++] = std::move(nuevoEstudiante);  
    return true;  
}
```

```
bool eliminar(const std::string& cedula) {  
    int idx = buscar(cedula);  
    if (idx == -1) return false;  
    for (int i = idx; i < cantidad - 1; i++) {  
        estudiantes[i] = std::move(estudiantes[i + 1]);  
    }  
}
```




```
estudiantes[--cantidad].reset();  
return true;  
}
```

El estudiante se guarda en la posición indicada por el contador cantidad, asegurando que la memoria se mantenga contigua y sin huecos intermediarios.

27. Paso 7: Instanciar Objetos

Java:

```
Estudiante nuevoEstudiante = new EstudiantePregrado(cedula, nombre, carrera, edad,  
promedio, activo, creditos, semestre);  
gestor.registrar(nuevoEstudiante);
```

C++:

```
auto estudiante_ = std::make_unique<EstudiantePregrado>(cedula, nombre, carrera, edad,  
promedio, activo, creditos, semestre);  
gestor.registrar(std::move(estudiante_));
```

La operación ejecuta tres acciones fundamentales:

- Creación e intermediación del objeto en memoria dinámica.
- Ejecución del constructor para inicializar atributos heredados y propios.
- Asignación de la referencia/puntero base para tratamiento polimórfico.

28. Tipos de datos utilizados

Variables	Tipo	Categoría	Descripción / Uso
edad, cantidad, creditosAprobados, semestre, horasInvestigacion	int	Primitivo	Valores numéricos enteros sin parte decimal.
promedio	double	Primitivo	Valores numéricos con coma flotante de doble precisión.
activo	boolean / bool	Primitivo	Valores lógicos (verdadero o falso).
cedula, nombre, carrera, tituloPrevio	String / std::string	Referencia / No primitivo	Cadenas de caracteres para texto.
estudiantes[]	Estudiante[] / unique_ptr[]	Referencia / No primitivo	Arreglo de referencias o punteros a objetos Estudiante.

29. Diagramas de clase

```
classDiagram  
class Estudiante {  
    #String cedula  
    #String nombre  
    #String carrera
```



```
#int edad
#double promedio
#bool activo
+mostrarInformacion()*
+getTipoEstudiante()*
}
class EstudiantePregrado {
    -int creditosAprobados
    -int semestre
    +mostrarInformacion()
}
class EstudiantePosgrado {
    -String tituloPrevio
    -int horasInvestigacion
    +mostrarInformacion()
}
class RegistroEstudiantes {
    -Estudiante[] estudiantes
    -int cantidad
    +registrar(Estudiante) bool
    +eliminar(String) bool
    +mostrarTodos()
}
Estudiante <|-- EstudiantePregrado
Estudiante <|-- EstudiantePosgrado
RegistroEstudiantes o-- Estudiante
```

30. Especificación del TDA

Estado:

- **estudiante:** Arreglo fijo de diez referencias o punteros a objetos de tipo Estudiante.
- **cantidad:** Número de posiciones actualmente ocupadas (entre 0 y 10).
- **CAPACIDAD:** Constante entera con valor fijado en 10.

Operaciones:

Método	Resultado / Descripción
registrar()	Agrega un estudiante si existe espacio disponible y la cédula no está duplicada.
mostrarTodos()	Presenta la información de todos los objetos registrados mediante polimorfismo.
buscarPorCedula()	Comprueba si una cédula ya existe y devuelve su índice numérico o -1.
modificar()	Actualiza el promedio y el estado activo/inactivo del estudiante en una posición específica.
eliminar()	Remueve un estudiante del arreglo y realiza la compactación contigua del vector.
estaLleno()	Indica si las diez posiciones del arreglo se encuentran totalmente ocupadas.



Invariantes del TDA:

1. $0 \leq \text{cantidad} \leq \text{CAPACIDAD}$.
2. Las posiciones desde 0 hasta cantidad - 1 contienen objetos válidos.
3. Las posiciones desde cantidad hasta CAPACIDAD - 1 están nulas/vacías (null o reset()).
4. No existen cédulas repetidas dentro del sistema.
5. La edad registrada es un valor positivo mayor a 0 (entre 15 y 100 años).
6. El promedio del estudiante está dentro del rango válido (promedio ≥ 0.0 y promedio ≤ 10.0).
7. Un estudiante de pregrado posee un número positivo de créditos aprobados.
8. Un estudiante de posgrado posee un número de horas de investigación mayor a 0 hrs.

31. Diagrama de flujo del programa principal

INICIO

Instanciar GestorEstudiantes gestor

Entero opcion

REPETIR

Mostrar "=== SISTEMA DE GESTIÓN DE ESTUDIANTES ==="

Mostrar "1. Registrar Estudiante (Pregrado / Posgrado)"

Mostrar "2. Mostrar todos los estudiantes"

Mostrar "3. Buscar estudiante por cédula"

Mostrar "4. Modificar estudiante"

Mostrar "5. Eliminar estudiante"

Mostrar "0. Salir"

Mostrar "Ingrese una opción: "

Leer opcion

SEGUN opcion HACER

Caso 1:

SI NO gestor.estaLleno() ENTONCES

Cadena cedula, nombre, carrera

Entero edad, tipo

Real promedio

Booleano activo

Leer cedula, nombre, carrera, edad, promedio, activo

SI gestor.buscarPorCedula(cedula) == -1 ENTONCES

Mostrar "Seleccione Tipo: 1. Pregrado | 2. Posgrado"

Leer tipo

SI tipo == 1 ENTONCES

Entero credits, semestre

Leer credits, semestre

Estudiante nuevo = Crear EstudiantePregrado(cedula, nombre, carrera,
edad, promedio, activo, credits, semestre)

gestor.registrar(nuevo)

SINO SI tipo == 2 ENTONCES

Entero horasInv

Cadena titulo

Leer horasInv, titulo

Estudiante nuevo = Crear EstudiantePosgrado(cedula, nombre, carrera,
edad, promedio, activo, horasInv, titulo)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
gestor.registrar(nuevo)
FIN SI
SINO
  Mostrar "Error: La cédula ya se encuentra registrada."
FIN SI
SINO
  Mostrar "Error: El registro se encuentra lleno."
FIN SI
```

Caso 2:
gestor.mostrarTodos()

Caso 3:
Cadena cedula
Leer cedula
Entero pos = gestor.buscarPorCedula(cedula)
SI pos != -1 ENTONCES
 Mostrar "Estudiante encontrado en la posición: ", pos
SINO
 Mostrar "Estudiante no registrado."
FIN SI

Caso 4:
Cadena cedula
Leer cedula
gestor.modificar(cedula)

Caso 5:
Cadena cedula
Leer cedula
gestor.eliminar(cedula)

Caso 0:
Mostrar "Saliendo del programa..."

De Otro Modo:
Mostrar "Opción no válida."
FIN SEGUN

HASTA QUE opcion == 0
FIN

32. Prueba de escritorio

Paso	Operación	Estado Estudiante	Estado Estudiante	Estado de Curso	Resultado de la prueba
1	Insertar Estudiante 1	Ced: 1801, Notas: []	Vacio	numEstudiantes = 1	Estudiante registrado con ID autonumérico 1.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



2	Agregar Nota 10.0 a E1	Ced: 1801, Notas: [10.0]	Vacio	numEstudiantes = 1	Se agrega la nota 1 (1/7) al estudiante 1.
3	Insertar Estudiante 2	Ced: 1801, Notas: [10.0]	Ced: 1802, Notas: []	numEstudiantes = 2	Estudiante registrado con ID autonumérico
4	Agregar Nota 8.0 a E2	Ced: 1801, Notas: [10.0]	Ced: 1802, Notas: [8.0]	numEstudiantes = 2	Se agrega nota a E2. Promedio E2 = 8.0.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☒ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Diseñar las clases Vehiculo, Automovil y Motocicleta con sus atributos para establecer la estructura orientada a objetos.
- Implementar el vector estático en RegistroVehiculos para encapsular y gestionar el almacenamiento de hasta 10 elementos.
- Aplicar el polimorfismo en mostrarInformacion() para procesar los datos recorriendo el vector de objetos.

2.10 Recomendaciones

- Extender las clases derivadas agregando nuevos tipos de vehículos dentro de la estructura orientada a objetos.
- Migrar el vector estático a una lista dinámica para superar la capacidad fija del almacenamiento en el TDA.
- Conservar el uso de polimorfismo al implementar nuevas operaciones que recorran el vector de objetos.

2.11 Referencias bibliográficas

- [1] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," Baeldung, may 2026. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>
- [2] "The Cost of Garbage Collection for State Machine Replication," *arXiv preprint*, arXiv:2405.11182, 2024. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182>.



2.12 Anexos

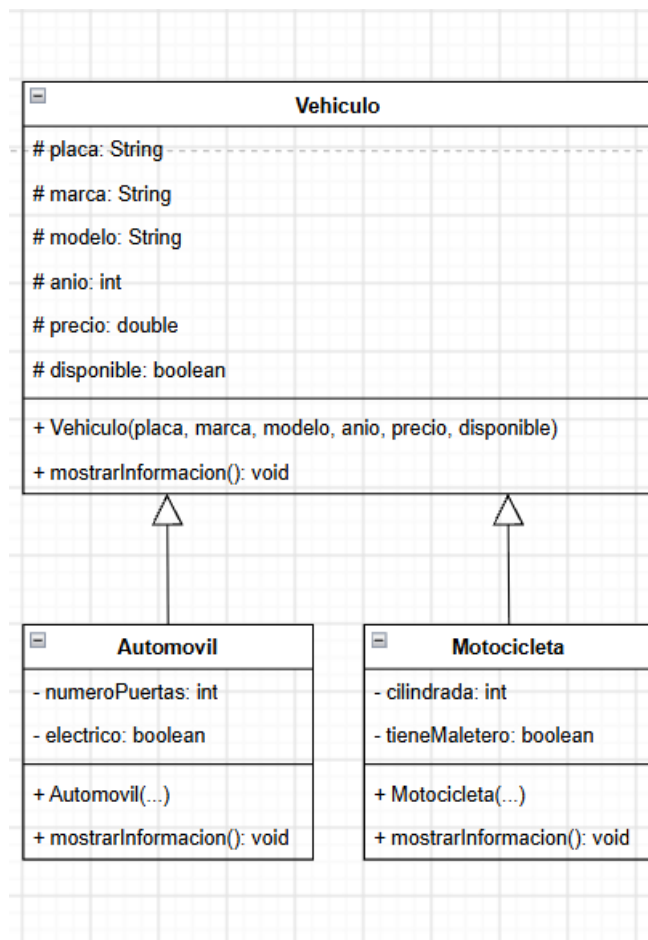


Figura 1. Diagrama de la clase *Vehiculo*, *Automovil* y *Motocicleta*.

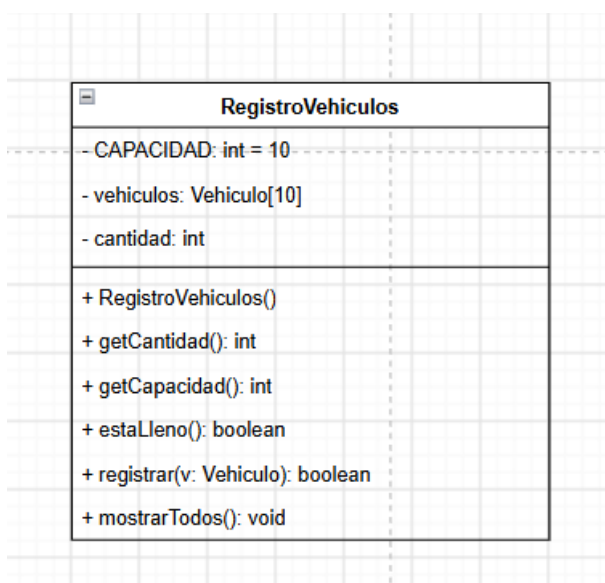


Figura 2. Diagrama clase *RegistroVehiculos*.

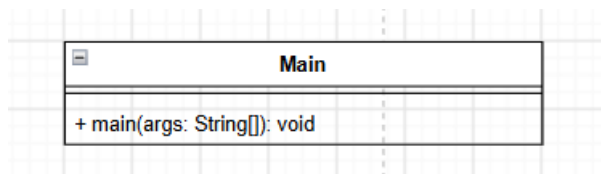


Figura 3. Diagrama de la clase Main.

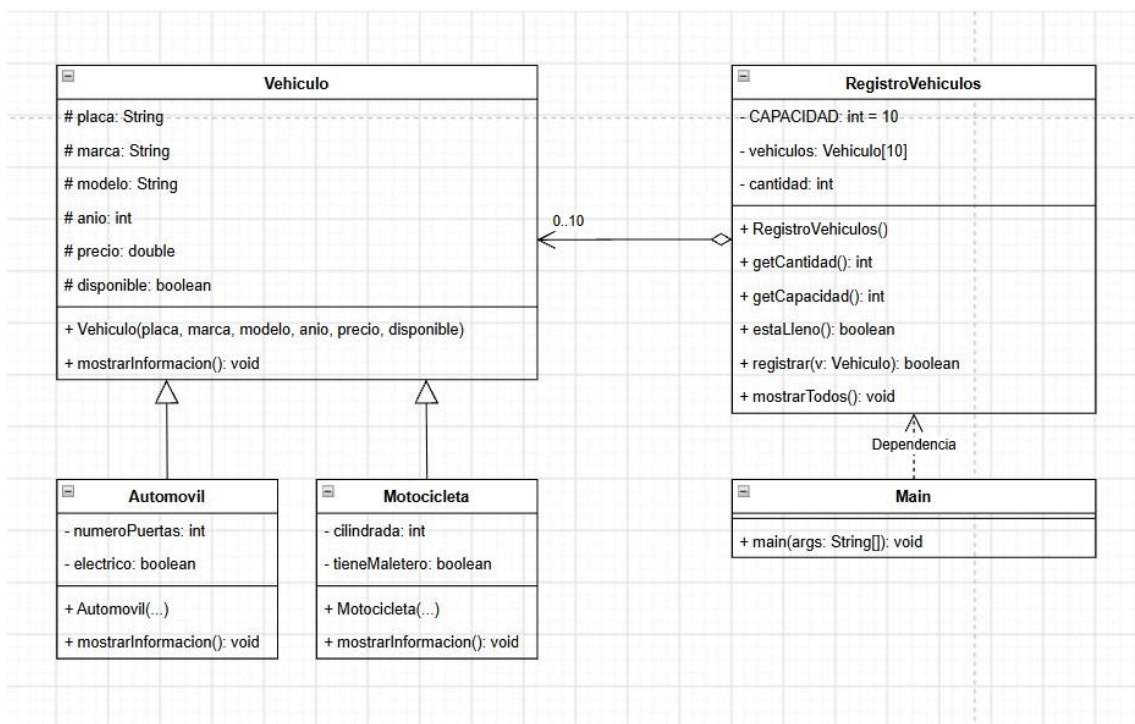


Figura 4. Diagrama de clases completo del TDA Vehiculo.



```
Disponible? (1: S / 2: N): s
Cilindrada (cc): 6
ERROR: El valor debe estar entre 50 y 2500.
Cilindrada (cc): 2500
Tiene maletero? (1: S / 2: N): n
Motocicleta registrada exitosamente.
```

```
Desea registrar otra motocicleta? (S/N): n
```

```
=====
          SISTEMA DE GESTION VEHICULAR
=====
```

```
1. Registrar Automovil
2. Registrar Motocicleta
3. Listar todos los vehiculos
4. Modificar vehiculo por placa
5. Eliminar vehiculo por placa
6. Salir
Seleccione una opcion (1-6): 3
```

```
[Registro 1]
```

```
-----
Tipo: Automovil
Placa: abc123
Marca: Honda
Modelo: Nose
Año: 2001
Precio: $2555.0
Disponible: No
Numero de Puertas: 6
Electrico: No
-----
```

```
[Registro 2]
```

```
-----
Tipo: Motocicleta
Placa: kjs5
Marca: 'sepa
Modelo: yamaha
Año: 2026
Precio: $5000.0
Disponible: Si
```

Figura 5. Ejecución en Java.



```
3. Listar todos los vehiculos
4. Modificar vehiculo por placa
5. Eliminar vehiculo por placa
6. Salir
Seleccione una opcion [1-6]: 1

--- REGISTRO DE AUTOMOVIL ---
Placa: abc123
Marca: VehiculoPoderoso
Modelo: 555sa
Anio: 20222
Precio: 2666
Disponible? (1: Si / 2: No): n
ERROR: Ingrese 1 para Si o 2 para No.
Disponible? (1: Si / 2: No): 2
Numero de Puertas: 5
Es electrico? (1: Si / 2: No): 2
Automovil registrado exitosamente.

Desea registrar otro automovil? (S/N): n

=====
          SISTEMA DE GESTION VEHICULAR
=====

1. Registrar Automovil
2. Registrar Motocicleta
3. Listar todos los vehiculos
4. Modificar vehiculo por placa
5. Eliminar vehiculo por placa
6. Salir
Seleccione una opcion [1-6]: 3

===== LISTADO DE VEHICULOS =====

[Registro 1]
-----
Tipo: Automovil
Placa: abc123
Marca: VehiculoPoderoso
Modelo: 555sa
Ano: 20222
Precio: $2666
Disponible: No
Numero de Puertas: 5
Electrico: No
```

Figura 6. Ejecución en C++.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

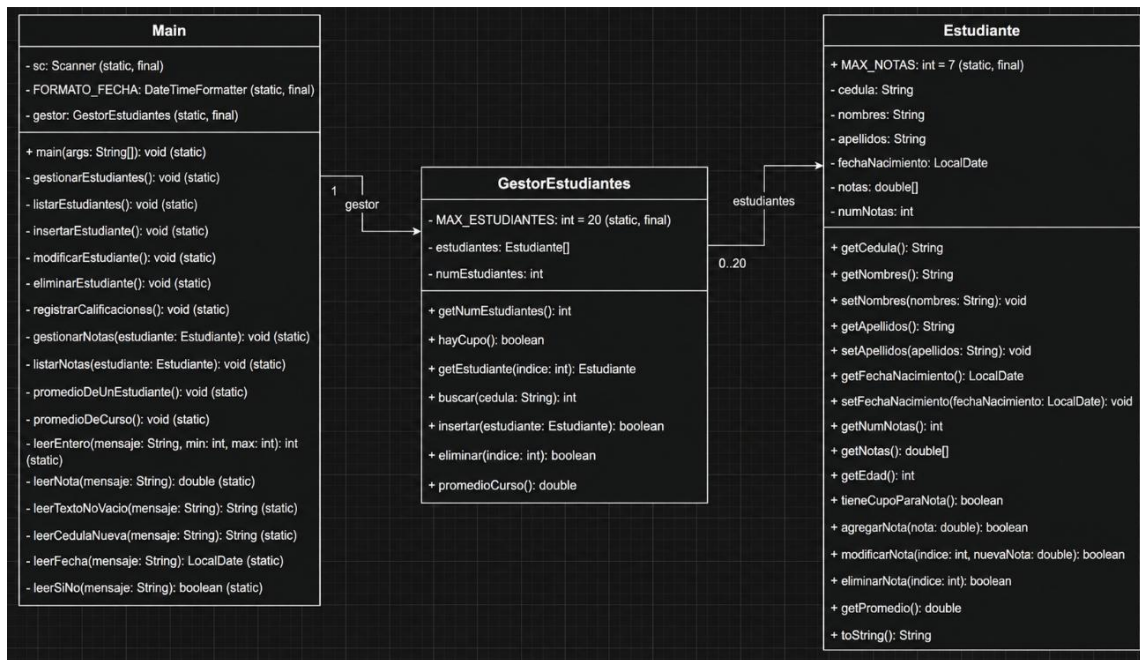


Figura 7. UML TDA Estudiantes.

```
2.- Registro de calificaciones.
3.- Determinar el promedio de notas de un estudiante.
4.- Determinar el promedio de notas del curso.
0.- Salir.
Ingrese su opcion (0-4): 1

=== ESTUDIANTES ===
No hay estudiantes registrados.
1.- Insertar.
2.- Modificar.
3.- Eliminar.
4.- Volver al menu principal.
Ingrese su opcion (1-4): 1
Ingrese la cedula del estudiante: 1850659864
Ingrese los nombres: Darwin Joel
Ingrese los apellidos: Tisalema Guashco
Ingrese la fecha de nacimiento (dd/MM/yyyy): 03/03/2007
Estudiante registrado correctamente.
¿Desea insertar otro estudiante? (S/N): n

=== ESTUDIANTES ===
1. Cedula: 1850659864 | Nombres: Darwin Joel | Apellidos: Tisalema Guashco | Fecha nac.: 3/3/2007 | Edad: 19
1.- Insertar.
2.- Modificar.
3.- Eliminar.
4.- Volver al menu principal.
Ingrese su opcion (1-4): 2
1. Cedula: 1850659864 | Nombres: Darwin Joel | Apellidos: Tisalema Guashco | Fecha nac.: 3/3/2007 | Edad: 19
Ingrese el numero del estudiante a modificar: 1
Datos actuales: Cedula: 1850659864 | Nombres: Darwin Joel | Apellidos: Tisalema Guashco | Fecha nac.: 3/3/2007 | Edad: 19
Nuevos nombres: Joel
Nuevos apellidos: Tisalema
Nueva fecha de nacimiento (dd/MM/yyyy): 03/05/2008
Estudiante actualizado correctamente.
¿Desea modificar otro estudiante? (S/N): n

=== ESTUDIANTES ===
1. Cedula: 1850659864 | Nombres: Joel | Apellidos: Tisalema | Fecha nac.: 3/5/2008 | Edad: 18
1.- Insertar.
2.- Modificar.
3.- Eliminar.
4.- Volver al menu principal.
Ingrese su opcion (1-4): 3
1. Cedula: 1850659864 | Nombres: Joel | Apellidos: Tisalema | Fecha nac.: 3/5/2008 | Edad: 18
Ingrese el numero del estudiante a eliminar: 2
Valor fuera de rango. Debe estar entre 1 y 1.
Ingrese el numero del estudiante a eliminar: |
```

Figura 8. Código TDA estudiantes C++.



```
Teclee su opción (0-4): 2
Ingrese la cédula del estudiante: 1805163282
Nombres: Alex | Apellidos: Mendoza | Edad: 19
No hay calificaciones registradas.
1.- Insertar calificación.
2.- Modificar calificación.
3.- Eliminar calificación.
4.- Volver al menú principal.
Teclee su opción (1-4): 1
Ingrese la calificación: 10
1. 10,00
1.- Insertar calificación.
2.- Modificar calificación.
3.- Eliminar calificación.
4.- Volver al menú principal.
Teclee su opción (1-4): 4

=== GESTOR DE PERSONAS ===
1.- Estudiantes.
2.- Registro de calificaciones.
3.- Determinar el promedio de notas de un estudiante.
4.- Determinar el promedio de notas del curso.
0.- Salir.
Teclee su opción (0-4): 3
Ingrese la cédula del estudiante: 1805163282
Nombres: Alex
Apellidos: Mendoza
Edad: 19
Promedio de calificaciones: 10,00

=== GESTOR DE PERSONAS ===
1.- Estudiantes.
2.- Registro de calificaciones.
3.- Determinar el promedio de notas de un estudiante.
4.- Determinar el promedio de notas del curso.
0.- Salir.
```

Figura 9. Código TDA estudiantes Java.

A continuación, se adjunta el enlace al repositorio que contiene el código fuente en C++ y Java, las capturas de pantalla de la ejecución, el diagrama UML y este documento en formato .pdf.

Enlace repositorio: <https://github.com/Damienq7w/ESTRUCTURA-DE-DATOS---APE-1.git>